



HKT: Hierarchical structure-based knowledge tracing

Qing Li ^{a,b}, Zhijun Huang ^{a,b}, Jianwen Sun ^{a,b}, Xin Yuan ^{a,b},
Shengyingjie Liu ^{a,b,*}, Zhonghua Yan ^{a,b}

^a National Engineering Research Center of Educational Big Data, Central China Normal University, Wuhan 430079, China

^b Faculty of Artificial Intelligence in Education, Central China Normal University, Wuhan 430079, China

ARTICLE INFO

Keywords:

Knowledge tracing
Graph representation learning
Graph contrastive learning
Hierarchical modeling
Educational data mining

ABSTRACT

Knowledge tracing (KT) is a fundamental task in Intelligent Tutoring Systems, aiming to predict learners' performance on specific questions and trace their evolving knowledge state. With the advancement of deep learning in this field, various methods have been applied to model the relations between knowledge. However, most existing knowledge tracing methods focus on modeling knowledge at a single level, neglecting the inherent hierarchical structure of knowledge, which limits their ability to capture complex relations. In this paper, we propose a novel hierarchical knowledge tracing model (HKT), which integrates influences of multiple knowledge levels to predict learners' performance. Specifically, we construct different types of hierarchical graphs to capture both intra-hierarchy dependencies and cross-hierarchy relations. To effectively combine information from multiple levels, we design weight allocation networks that dynamically assign weights to different knowledge levels, thereby synthesizing their effects for accurate performance prediction. Experimental results demonstrate that HKT outperforms baseline methods on multiple benchmark datasets, validating the effectiveness of integrating knowledge across all levels compared to single-level models.

1. Introduction

The rapid expansion of online learning platforms has generated an immense volume of learner behavior data, opening up new opportunities for the development of Intelligent Tutoring Systems (ITS) (Wang, Lai and Huang, 2024). Effectively managing and analyzing this data, as well as accurately assessing a learner's knowledge state, continues to pose significant challenges for ITS. Knowledge Tracing (KT), which draws on educational theory and leverages data-driven methodologies, is designed to model cognitive progression and dynamically infer learners' knowledge states from historical learning data (Abdelrahman, Wang, & Nunes, 2023; Qian, Zheng, Wang, & Li, 2024). It provides valuable insights and practical tools for making decisions and managing in education, serving as a powerful driver for personalized learning (Wang & Sahebi, 2023).

Early KT models based on Hidden Markov Models treat each knowledge concept as an independent variable (Falakmasir, Pardos, Gordon, & Brusilovsky, 2013), which can be considered a pointwise model. We refer to it as Pointwise Knowledge Tracing (POI-KT), as shown in Fig. 1(a). Although the approaches are relatively straightforward to implement, they neglect the interdependencies between knowledge concepts, often leading to limited effectiveness in applications (Liu, Hu, Bu, & Yu, 2022). The emergence of deep learning technologies motivates researchers to employ more advanced neural network models to enhance the predictive abilities of KT (Yang, Yu, Ma, Zhang, & Li, 2022). Whether the backbone network is based on recurrent neural networks (Piech et al., 2015) or

* Corresponding author.

E-mail addresses: viven_a@ccnu.edu.cn (Q. Li), zhijunhuang@mails.ccnu.edu.cn (Z. Huang), sunjw@ccnu.edu.cn (J. Sun), yuanxin@mails.ccnu.edu.cn (X. Yuan), lsyj@mails.ccnu.edu.cn (S. Liu), hit_yan@mail.ccnu.edu.cn (Z. Yan).

<https://doi.org/10.1016/j.ipm.2025.104206>

Received 22 January 2025; Received in revised form 20 April 2025; Accepted 1 May 2025

Available online 19 May 2025

0306-4573/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

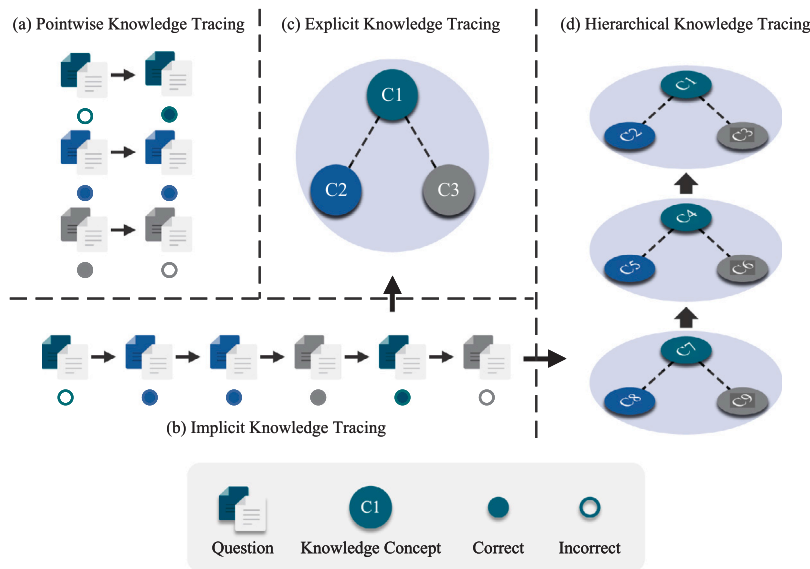


Fig. 1. The 4 types of KT models. (a) KT with independent modeling of knowledge concepts; (b) KT with implicit modeling of knowledge concepts; (c) KT with explicit modeling of knowledge concepts; (d) KT with hierarchical modeling of knowledge concepts.

self-attention networks (Ghosh, Heffernan, & Lan, 2020), the hidden layers of these models inherently consider the latent connections between knowledge concepts during parameter optimization, enabling them to predict future responses effectively. They do not explicitly depict the associations among knowledge concepts, relying instead on internal parameters for implicit learning, which we classify as Implicit Knowledge Tracing (IMP-KT), as shown in Fig. 1(b). Although they show some improvement in predictive accuracy, their flexibility is limited in contexts with complex knowledge structures. After that, researchers have applied graph models to represent the relations between knowledge concepts (Huang et al., 2024), significantly enhancing predictive performance in specific contexts. We refer to these models as Explicit Knowledge Tracing (EXP-KT), as depicted in Fig. 1(c), which effectively captures intricate dependencies among concepts, thereby improving the accuracy of learner behavior predictions.

Current research on modeling learners' knowledge structures has predominantly focused on single-level, planar relations between knowledge concepts, without fully considering the complexity of the hierarchical organization and multi-level collaboration of knowledge concepts revealed by cognitive architecture theory (Sweller, Van Merriënboer, & Paas, 1998). For example, when a learner answers a question related to “finding a common denominator”, they not only need to understand this concept but also master the higher-level concept of the “basic properties of fractions”. Specifically, in order to solve this problem, the learner must first understand the “basic properties of fractions” in order to identify that the question is testing “finding a common denominator” rather than “simplifying a fraction” and then apply the relevant knowledge to answer correctly. Therefore, multi-level modeling of knowledge concepts is crucial for accurately assessing learners' knowledge structures.

Guided by cognitive architecture theory (Sweller et al., 1998), learners' cognitive state modeling with the hierarchical structure of knowledge concepts involves tackling two key challenges. (1) The difference in the level of abstraction of various knowledge concepts makes it a significant challenge to clearly define the hierarchical structure of these concepts within a unified framework. (2) Different levels of knowledge have varying impacts on learner responses, making it necessary to assess the influence weight of each level. However, existing knowledge tracing (KT) methods typically focus on modeling a single level, neglecting the synergistic effects between different levels, which in turn affects the model's prediction accuracy and generalization ability. Addressing these challenges is vital for developing effective models that accurately capture and utilize the hierarchical nature of knowledge concepts.

In this work, we propose the Hierarchical Knowledge Tracing (HKT) model, which explicitly incorporates the hierarchical structure of knowledge concepts. It provides a more nuanced understanding of learner performance, distinguishing it from existing methods, as depicted in Fig. 1(d). For (1), we introduce a **Knowledge Hierarchy Tree (KHT)** based on expert prior knowledge, which models the vertical hierarchical relationships between knowledge concepts and establishes a top-down knowledge organization framework. Meanwhile, we construct a **Knowledge Association Graph (KAG)** using learning sequence information to represent interrelations between concepts at the same hierarchical level. Subsequently, we construct hierarchical embedding representations that not only indicate the level of each knowledge concept but also capture the distances between levels. These embeddings are further refined through the application of a cross-graph, hierarchical contrastive learning strategy. For (2), we propose **weight allocation networks** that can dynamically assess the impact of different levels of knowledge concepts on prediction results. By adaptively assigning different levels of weights based on the relevance of the prediction task, the model can provide a comprehensive evaluation of the learner's responses. Experimental results based on real-world datasets show a significant improvement in predictive performance. Furthermore, our approach exhibits strong interpretability. The explicit modeling of knowledge concepts using KHT and KAG provides an interpretable and structured knowledge framework that aligns with human cognition. The dynamic weight

allocation mechanism also offers instructors a clear explanatory basis for accurately identifying learners' weak knowledge levels. The principal contributions of this work are summarized as follows:

1. We combine knowledge tracing with a hierarchical knowledge structure model, moving beyond traditional single-level flat models to establish a multi-level, hierarchical framework.
2. We introduce the KHT and KAG, which model the relations between knowledge concepts across different levels and within the same level. These are integrated into a cross-graph, hierarchical contrastive learning framework to improve the model's representation ability.
3. We design the weight allocation networks that dynamically assign weights to knowledge concepts at different levels, integrating these weighted concepts to provide a comprehensive prediction of learner responses.
4. Experiments on real-world datasets show that our model outperforms other methods, achieving superior predictive performance over baseline models.

2. Related work

2.1. Knowledge tracing

Knowledge Tracing is a fundamental task in intelligent tutoring systems, aiming to dynamically infer a learner's knowledge state based on their learning interactions (Jiang, Wei, Zhang, & Zhang, 2024; Wang & Sahebi, 2023; Zhao, Xia, Jiang, & He, 2023). With the rapid accumulation of educational data and advancements in deep learning techniques, KT methods have evolved from traditional statistical models to sophisticated deep learning-based approaches (Wang et al., 2023). Based on different strategies for modeling knowledge concepts, We categorize KT methods into three main types: Pointwise Knowledge Tracing (POI-KT), Implicit Knowledge Tracing (IMP-KT), and Explicit Knowledge Tracing with relations Modeling (EXP-KT).

POI-KT was a key focus in the early development of KT methods, with Bayesian Knowledge Tracing (BKT) (Corbett & Anderson, 1994) serving as a notable example. These approaches treat each knowledge concept as an independent variable and employ a Bayesian probabilistic framework to update a learner's mastery of specific knowledge over time (Bulut, Shin, Yildirim-Erbasli, Gorgun, & Pardos, 2023). While POI-KT methods are both simple and easy to interpret, they rest on the assumption that knowledge concepts are entirely independent. This simplification disregards the intricate relations that exist within complex knowledge networks, thereby limiting their capacity to handle interconnected knowledge structures effectively. Consequently, their predictive accuracy tends to be suboptimal in scenarios involving more sophisticated knowledge interdependencies.

With the rapid advancement of deep learning technologies, researchers have developed increasingly sophisticated models to capture the intricate relations between learners' behaviors and underlying knowledge concepts (Casalino et al., 2021). Within the framework of IMP-KT, models such as DKT (Piech et al., 2015) and AKT (Ghosh et al., 2020) have emerged as exemplary approaches. DKT employs Recurrent Neural Networks (Zaremba, 2014) to model learners' sequential interactions, effectively capturing personalized learning trajectories. However, it relies on implicit network architectures to infer associations among knowledge concepts, lacking explicit modeling of these relations, which can constrain its performance. AKT (Ghosh et al., 2020) utilizes multi-layer attention mechanisms (Waswani et al., 2017) to model the mastery and forgetting dynamics of knowledge concepts, thereby enhancing the predictive performance of knowledge tracing. Despite these implicit association modeling techniques leverage deep learning to achieve performance enhancements, the absence of clear representations of knowledge relations restricts their effectiveness, particularly in educational environments characterized by complex hierarchical knowledge structures.

Additionally, researchers have introduced EXP-KT methods that leverage graph neural networks to explicitly model relations between knowledge concepts. One prominent example is GKT (Nakagawa, Iwasawa, & Matsuo, 2019), which models questions and knowledge concepts as nodes and employs graph neural networks to capture their relations. While GKT models knowledge relations at a single level, it struggles to represent the multi-level hierarchical structures found in real-world knowledge. Most existing EXP-KT methods focus primarily on flat or same-level relations, which limits their effectiveness in scenarios involving complex knowledge hierarchies.

2.2. Graph representation learning

Representation learning seeks to extract meaningful feature representations from raw data through automated methods, enabling their application across a range of downstream tasks (Bengio, Courville, & Vincent, 2013). As a significant branch of representation learning, Graph Representation Learning (GRL) (Hamilton, 2020) focuses on embedding nodes, edges, and their attributes from graph structures into a vector space, while preserving the graph's topology and semantic information (Chen, Wang, Wang, & Kuo, 2020; Li, Jian et al., 2024). These embeddings not only capture the relations between nodes effectively but also support various tasks such as classification, clustering, and link prediction.

Early approaches to graph representation learning predominantly relied on random walk techniques and matrix factorization. Notable examples include DeepWalk (Perozzi, Al-Rfou, & Skiena, 2014) and node2vec (Grover & Leskovec, 2016), which simulate random walks to generate sequences of nodes and employ the Skip-gram model to learn node embeddings. These methods effectively capture structural information by maximizing local neighborhood similarity. Furthermore, techniques such as negative sampling and noise contrastive estimation (NCE) (Gutmann & Hyvärinen, 2010) are widely adopted to optimize embeddings, improving their robustness and discriminative power.

Table 1
The key mathematical notations and descriptions.

Notation	Description
S, Q, C	The set of students, questions and concepts
Z, M, N	The total number of students, questions and concepts
q_i, r_i	The question attempted by the learner and the corresponding outcome at time step i
$\mathcal{T}, \mathcal{G}$	The knowledge hierarchy tree and the knowledge association graph
$\mathcal{V}, \mathcal{E}, \mathcal{I}$	The set of nodes, edges, and hierarchical levels
$\mathbf{V}, \mathbf{L}$	The embedding matrix of nodes and the embedding matrix of hierarchical levels
$\mathbf{H}, \mathbf{h}$	The node embeddings integrated with hierarchical information
$\mathcal{P}, p$	The path from a leaf node to the root node
$w^i, \hat{p}^i$	The weight and predicted value of the knowledge concept at the i th level

With the development of deep learning, GCL has become a key direction in graph representation learning (Khoshraftar & An, 2024). GCL focuses on learning robust and semantically rich node embeddings by constructing positive and negative sample pairs and employing contrastive loss functions, such as InfoNCE loss (Oord, Li, & Vinyals, 2018). Notable approaches, such as GraphCL (You et al., 2020) and Deep Graph Infomax (Veličković et al., 2018), enhance the performance and generalization of graph representation learning through diverse data augmentation strategies and contrastive objectives (Zhang, Niu, Ma, & Zhang, 2025).

In KT tasks, GRL has been employed to model the complex relations between knowledge concepts (Tong et al., 2020), thereby improving the accuracy of inferring learners' knowledge states. Recently, KT methods based on graph contrastive learning have gained prominence, utilizing contrastive learning strategies to further enhance embedding quality and model generalization (Wang, Ma, Zhao and Li, 2024). However, most existing approaches focus primarily on flattened graph structures, neglecting the hierarchical organization of knowledge concepts, which restricts their effectiveness in scenarios with complex knowledge structures.

3. Preliminaries

This section presents the preliminary knowledge, including the key definitions and notations required throughout this paper.

Knowledge tracing (KT) is a critical technique in the development of intelligent education systems, enabling the dynamic evaluation and prediction of a learner's mastery of knowledge over time. Let the set of learners be denoted as $S = \{s_1, s_2, \dots, s_Z\}$, where $|S| = Z$, and the set of questions as $Q = \{q_1, q_2, \dots, q_M\}$, where $|Q| = M$. The primary goal of knowledge tracing is to model a learner's historical interaction sequence $\mathcal{X}_s = \{x_1^s, x_2^s, \dots, x_t^s\}$, in order to predict their performance on a future question at the next time step $t+1$. Each interaction x_t^u is composed of a question q_i and the corresponding learner's response r_i , where $r_i = 1$ indicates a correct answer and $r_i = 0$ otherwise. The objective is to estimate the probability that a learner will correctly answer a given question q_{t+1} at time $t+1$, conditioned on their historical interactions. The probability is formally expressed as: $p(q_{t+1} = 1 \mid \mathcal{X}_s, q_{t+1})$.

A knowledge concept is denoted as $c \in C$, where $C = \{c_1, c_2, \dots, c_N\}$ represents a set of N distinct knowledge concepts. In general, these knowledge concepts are structured hierarchically to represent relations between them. This hierarchical structure can be formally modeled as a Knowledge Hierarchy Tree $\mathcal{T} = (\mathcal{V}, \mathcal{E}, \mathcal{I})$, where $\mathcal{V}$ represents the set of nodes, with each node $v \in \mathcal{V}$ uniquely corresponding to a knowledge concept in C , $\mathcal{E}$ represents the set of edges, with each edge $e_{uv} \in \mathcal{E}$ connecting two nodes u and v , $\mathcal{I}$ represents the levels of the hierarchy, where each level $i \in \mathcal{I}$ specifies the depth of nodes within the tree. To quantify the hierarchical structure, a mapping $\phi : \mathcal{V} \rightarrow \mathcal{I}$ is defined, such that $\phi(v)$ provides the hierarchical level of node v . By convention, the root node of the tree is assigned a level of 1.

To capture relation between knowledge concepts at the same hierarchical level, we introduce the Knowledge Association Graph. For the k th level of the hierarchy, the graph is defined as $\mathcal{G}^k = (\mathcal{V}^k, \mathcal{E}^k)$, where $\mathcal{V}^k = \{v \in \mathcal{V} \mid \phi(v) = k\}$ is the set of nodes at the k th level, $\mathcal{E}^k$ is the set of edges representing relations between these nodes. The edges $\mathcal{E}^k$ are constructed based on semantic dependencies, derived from the co-occurrence patterns of knowledge concepts in learners' response sequences. By analyzing how knowledge concepts appear together in responses, we identify and quantify their relations, which are then represented as edges in the graph. The Knowledge Association Graph consists of multiple subgraphs, each corresponding to a specific hierarchical level. The complete graph is expressed as $\mathcal{G} = \{\mathcal{G}^1, \mathcal{G}^2, \dots, \mathcal{G}^I\}$, where $\mathcal{G}^k$ is the subgraph for the k th level, and I is the total number of levels. For clarity, the key notations used are summarized in Table 1.

4. Methodology

As shown in Fig. 2, HKT begins by modeling the hierarchical relations within the KHT and the associative relations between knowledge concepts in the KAG. It then employs a hierarchy-oriented contrastive learning strategy to generate knowledge concept embeddings that effectively capture hierarchical information. Finally, a weight allocation network is used to accurately assess the influence of knowledge concepts at different levels on prediction outcomes and to allocate their weights in a rational manner for the final prediction task.

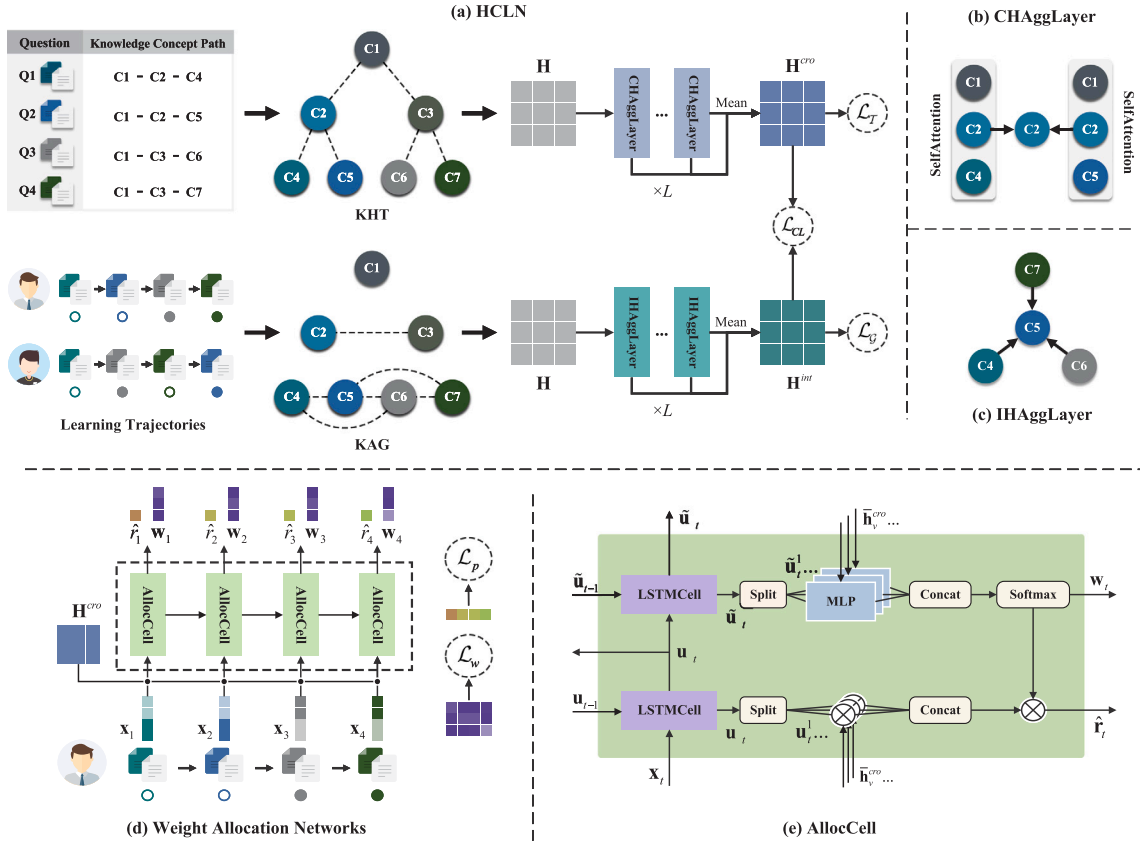


Fig. 2. The main frame work of HKT. Subfigure (a) The construction of the KHT and KAG is followed by embedding training through graph reconstruction and cross-graph hierarchical contrastive learning. (d) The structure of the weight allocation networks. (b) The details of the cross-hierarchy encoding layer. (c) The specifics of the intra-hierarchy encoding layer. (e) The computational process of AllocCell.

4.1. Embeddings

In our work, the initial feature matrix $\mathbf{H}$ for knowledge concepts is derived from the combination of the embeddings $\mathbf{V}$ of the corresponding nodes and the hierarchical embeddings $\mathbf{L}$ of these nodes:

$$\mathbf{H} = \mathbf{V} + \mathbf{L}, \quad (1)$$

where $\mathbf{V} \in \mathbb{R}^{|P| \times d}$ represents the learnable parameters, $\mathbf{L} \in \mathbb{R}^{|P| \times d}$ denotes the hierarchical embeddings corresponding to the nodes. This integration ensures the resulting node embeddings effectively capture hierarchical information, which is crucial for connecting KHT and KAG.

Knowledge concepts at adjacent levels generally exhibit greater semantic similarity compared to those at more distant levels within the hierarchy. As such, hierarchical embeddings should reflect the similarity across different levels of the hierarchy. According to Bochner's theorem (Loomis, 2013), any shift-invariant kernel function can be expressed as the Fourier transform of a non-negative measure. Building on this, we construct the hierarchical embeddings for the i th level using feature representations derived from the Fourier transform, as follows:

$$\mathbf{I}_i = \sqrt{\frac{1}{d_i}} [\cos(\omega_1 i), \sin(\omega_1 i), \dots, \cos(\omega_d i), \sin(\omega_d i)], \quad (2)$$

where $\omega = [\omega_1, \omega_2, \dots, \omega_d]$ denotes the learnable parameters, and d represents the dimensionality of the hierarchical embeddings. The hierarchical embedding $\mathbf{L}$ is formulated by incorporating the learnable frequency parameters ω , allowing it to approximate a shift-invariant kernel and thereby satisfy the requirements of Bochner's theorem (Loomis, 2013). This approach ensures that the hierarchical embeddings exhibit smooth and continuous variations with shift-invariant properties in the embedding space, enabling better continuity between adjacent hierarchical levels.

The similarity between hierarchical embeddings is defined as:

$$\psi(i - j) = \mathbf{L}_i \cdot \mathbf{L}_j = \frac{1}{d} \sum_{k=1}^d \cos(\omega_k(i - j)) \propto \cos(\omega(i - j)). \quad (3)$$

If the final values of ω_k are identical or produce equivalent effects, the similarity between hierarchical embeddings becomes directly proportional to the cosine function $\cos(\omega_k(i - j))$ at a single frequency. On the other hand, if the final values of ω_k vary across frequencies, the hierarchical embeddings represent a weighted average of cosine functions over multiple frequencies. It can also be viewed as a smoothed approximation of $\cos(\omega_k(i - j))$. This effectively encoding the hierarchical embeddings and accurately capturing the distance relations between different hierarchical levels.

4.2. Cross-hierarchy aggregation networks

We propose a hierarchical encoding method based on path sequences to effectively capture the information of nodes within a hierarchical structure. The sequence formed by traversing from leaf nodes to the root node in the knowledge hierarchy tree is regarded as a hierarchical path. Using a self-attention mechanism, we construct sequential features for the nodes along the path, enabling the encoding of hierarchical relations between them.

Consider any leaf node $v \in \mathcal{V}^l$. There exists at least one path $\mathcal{P}$ from v to the root node. A specific path $\mathcal{P}$, denoted as $\mathcal{P} = \{p_I, p_{I-1}, \dots, p_1\}$, satisfies $p_I = v$ and $p_1 = h_v$. Additionally, for any $1 \leq i < I$, it holds that $e_{p_i, p_{i+1}} \in \mathcal{E}$. The embedding matrix associated with the path $\mathcal{P}$ is defined as:

$$\mathbf{P} = [\mathbf{p}_1^\top, \mathbf{p}_2^\top, \dots, \mathbf{p}_I^\top]^\top. \quad (4)$$

Sequential features for each node across different hierarchical levels are constructed using path data in conjunction with a multi-head attention mechanism:

$$\mathbf{Z}_i = \text{Attention}(\mathbf{P}\mathbf{W}_i^Q, \mathbf{P}\mathbf{W}_i^K, \mathbf{P}\mathbf{W}_i^V), \quad (5)$$

$$\mathbf{A} = \text{MultiHead}(\mathbf{P}) = [\mathbf{Z}_1 \parallel \mathbf{Z}_2 \parallel \dots \parallel \mathbf{Z}_{N_{\text{head}}}] \cdot \mathbf{W}^M, \quad (6)$$

where $\mathbf{W}^M \in \mathbb{R}^{d \times d}$ represents the weight matrix, and $\parallel$ denotes the concatenation operation. A residual connection is subsequently applied, resulting in the output path feature matrix:

$$\mathbf{O} = (\mathbf{P} + \text{Dropout}(\mathbf{A})) \mathbf{W}^o, \quad (7)$$

where $\mathbf{W}^o \in \mathbb{R}^{d \times d}$ represents the weight matrix. As multiple paths can converge on the root node, a given node may appear in the same position within several path sequences.

To address this, we calculate the attention weights for each occurrence of the node across all paths:

$$\alpha_v^{\mathcal{P}} = \frac{\exp(\mathbf{a} \cdot [\mathbf{h}_v \parallel \mathbf{o}_v^{\mathcal{P}}])}{\sum_{\mathcal{P}^* \in S_v} \exp(\mathbf{a} \cdot [\mathbf{h}_v \parallel \mathbf{o}_v^{\mathcal{P}^*}])}, \quad (8)$$

where $\mathbf{a} \in \mathbb{R}^{2d}$ denote the parameter vector, and S_v represent the set of all paths that pass through node v . $\mathbf{o}_v^{\mathcal{P}}$ refers to the output feature of node v in path $\mathcal{P}$. The features of node v are updated using the attention weights as follows:

$$\mathbf{h}_v^{\text{cro}} \leftarrow \sum_{\mathcal{P} \in S_v} \alpha_v^{\mathcal{P}} \mathbf{o}_v^{\mathcal{P}}, \quad (9)$$

where $\mathbf{h}_v^{\text{cro}} \in \mathbb{R}^d$ denotes the final embedding of node v_i . Given that the encoding process spans multiple layers, the final feature representation is derived by averaging the outputs from all encoding layers:

$$\bar{\mathbf{h}}_v^{\text{cro}} = \text{Mean}(\mathbf{h}_v^{\text{cro},1}, \mathbf{h}_v^{\text{cro},2}, \dots, \mathbf{h}_v^{\text{cro},L}). \quad (10)$$

The features of all nodes in the set $\mathcal{V}$ can be compactly represented as the matrix $\bar{\mathbf{H}}^{\text{cro}}$, where each row of the matrix corresponds to the final embedding vector $\bar{\mathbf{h}}_v^{\text{cro}}$ of a node $v \in \mathcal{V}$.

4.3. Intra-hierarchy aggregation networks

In hierarchical knowledge structures, the relations between knowledge concepts within the same level are as important as the inter-level connections across different levels. Intra-hierarchy aggregation networks are employed to explore and represent intra-level relations, enabling the identification of connections within the same hierarchical level. It focuses on capturing the structural dependencies among knowledge concepts at the same level, ensuring a comprehensive understanding of their interconnections.

Consider a node v , with its neighboring nodes denoted by the set $\mathcal{V}_{\text{neg}}$, comprising all nodes directly connected to v . The attention coefficient, which quantifies the importance of the relations between v and its neighbors, is defined as:

$$\beta_{uv} = \mathbf{b}([\mathbf{W}\mathbf{h}_u \parallel \mathbf{W}\mathbf{h}_v]), \quad u \in \mathcal{V}_{\text{neg}}, \quad (11)$$

where $\mathbf{W} \in \mathbb{R}^{d \times d}$, $\mathbf{b} \in \mathbb{R}^{2d}$, and $\parallel$ indicates the concatenation operation. It is followed by the application of an activation function and a normalization process:

$$\alpha_{uv} = \frac{\exp(\text{LeakyReLU}(\beta_{uv}))}{\sum_{k \in \mathcal{V}_{\text{neg}}} \exp(\text{LeakyReLU}(\beta_{uk}))}. \quad (12)$$

The normalized attention weights are utilized to aggregate the features of neighboring nodes through a weighted summation. The outputs from all attention heads are subsequently concatenated to produce the final node representations:

$$\mathbf{h}_v^{int} \leftarrow \left\| \sum_{k=1}^K \sigma \left(\sum_{u \in \mathcal{V}_{neg}} \alpha_{uv}^k \mathbf{W}^k \mathbf{h}_u \right) \right\|, \quad (13)$$

where $\mathbf{h}_v^{int} \in \mathbb{R}^d$ denotes the intermediate embedding of node v , while K represents the number of attention heads and σ denotes the activation function.

Similarly, as the encoding process involves multiple layers, the final feature representation is obtained by averaging the outputs across all layers:

$$\bar{\mathbf{h}}_v^{int} = \text{Mean}(\mathbf{h}_v^{int,1}, \mathbf{h}_v^{int,2}, \dots, \mathbf{h}_v^{int,L}). \quad (14)$$

The features of all nodes in the set $\mathcal{V}$ can be compactly represented as the matrix $\bar{\mathbf{H}}^{int}$, where each row of the matrix corresponds to the final intermediate embedding $\bar{\mathbf{h}}_v^{int}$ of a node $v \in \mathcal{V}$.

4.4. Hierarchical contrastive learning

We propose a hierarchy-oriented knowledge contrastive learning strategy that effectively captures the information embedded within the KHT and the KAG. Graph reconstruction techniques are utilized to enhance the hierarchical properties. Our method addresses the limitations of single-graph representations and ensures consistent embeddings for identical knowledge concepts across different graphs by introducing a cross-graph hierarchical contrastive learning mechanism. By integrating complementary information from both the KHT and KAG, it enhances the coherence of the resulting embeddings.

A hierarchical structure loss is introduced within the KHT to reinforce the hierarchical properties of knowledge concepts. It ensures that parent-child relations in the tree are reflected with preserved similarity in the embedding space:

$$\mathcal{L}_{\mathcal{T}} = -\log(\sigma(s_{pos}(\bar{\mathbf{h}}_v^{cro}, \bar{\mathbf{h}}_u^{cro}))) - \frac{1}{|\mathcal{V}_{neg}|} \sum_{k \in \mathcal{V}_{neg}} \log(\sigma(-s_{neg}(\bar{\mathbf{h}}_v^{cro}, \bar{\mathbf{h}}_k^{cro}))), \quad (15)$$

where $\sigma(x)$ represents the sigmoid function, $s_{pos}(\bar{\mathbf{h}}_v^{cro}, \bar{\mathbf{h}}_u^{cro})$ denotes the score for positive samples, and $s_{neg}(\bar{\mathbf{h}}_v^{cro}, \bar{\mathbf{h}}_k^{cro})$ represents the score for negative samples. $\mathcal{V}_{neg}$ refers to the set of negative samples.

The scores are computed using dot products to measure the similarity between sample pairs:

$$s_{pos/neg}(\bar{\mathbf{h}}_v^{cro}, \bar{\mathbf{h}}_u^{cro}) = \frac{\bar{\mathbf{h}}_v^{cro} \cdot \bar{\mathbf{h}}_u^{cro}}{\tau}, \quad (16)$$

where τ denotes the temperature parameter, while $\cdot$ represents the dot product operation.

A similar enhancement strategy is applied to the KAG to further refine its structural representation and improve embedding quality:

$$\mathcal{L}_{\mathcal{G}} = -\log(\sigma(s_{pos}(\bar{\mathbf{h}}_v^{int}, \bar{\mathbf{h}}_u^{int}))) - \frac{1}{|\mathcal{V}_{neg}|} \sum_{k \in \mathcal{V}_{neg}} \log(\sigma(-s_{neg}(\bar{\mathbf{h}}_v^{int}, \bar{\mathbf{h}}_k^{int}))), \quad (17)$$

where $s_{pos}(\bar{\mathbf{h}}_v^{int}, \bar{\mathbf{h}}_u^{int})$ denotes the score for positive samples, $s_{neg}(\bar{\mathbf{h}}_v^{int}, \bar{\mathbf{h}}_k^{int})$ represents the score for negative samples, and $\mathcal{V}_{neg}$ represents the set of negative samples.

We introduce a cross-graph hierarchical contrastive learning strategy to overcome the shortcomings of single-graph representations. By drawing on complementary information from two graphs, It achieves better alignment of embeddings for identical knowledge concepts, thereby improving the consistency and dependability of the resulting knowledge representations.

$$\mathcal{L}_{CL} = -\frac{1}{N} \sum_{i=1}^N \log \left(\frac{\exp(\bar{\mathbf{h}}_v^{cro} \cdot \bar{\mathbf{h}}_v^{int} / \gamma)}{\sum_{j=1}^N \delta_{l_i, l_j} \exp(\bar{\mathbf{h}}_v^{cro} \cdot \bar{\mathbf{h}}_u^{int} / \gamma)} \right), \quad (18)$$

where γ is the scaling factor and serves as a hyperparameter. N denotes the total number of nodes, and δ_{l_i, l_j} is an indicator function that equals 1 if l_i and l_j belong to the same category, and 0 otherwise. The aim is to enable contrastive learning between knowledge concepts at the same hierarchical level, while concepts from different levels are embedded at greater distances in the representation space.

The overall optimization objective is defined as:

$$\mathcal{L}_{HCL} = \mathcal{L}_{\mathcal{T}} + \mathcal{L}_{\mathcal{G}} + \mathcal{L}_{CL}. \quad (19)$$

4.5. Weight allocation networks

In previous work, we derived embeddings for knowledge concepts across various hierarchical levels. However, the influence of these concepts on learner performance differs not only across levels but also over time. Consequently, determining appropriate

Algorithm 1 Training of embeddings

Input: $\mathcal{T}, \mathcal{G}, \mathcal{P}, \mathbf{V}, \mathbf{P}$
Output: The representations of $\bar{\mathbf{H}}^{cro}, \bar{\mathbf{H}}^{int}$

- 1: $\mathbf{H} \leftarrow \mathbf{V} + \mathbf{L}$
- 2: **for** layer = 1 **to** L **do**
- 3: Derive α_v according to Eq. 8
- 4: Update $\mathbf{h}_v^{cro}$ by weighted summation of nodes v in paths
- 5: Derive β_{nv} and α_{nv} according to Eq. 11 and Eq. 12
- 6: Update $\mathbf{h}_v^{int}$ by weighted summation of neighbor nodes
- 7: **end for**
- 8: Update $\mathbf{h}_v^{cro}, \mathbf{h}_v^{int}$ by weighted average of $\mathbf{h}_v^{cro}, \mathbf{h}_v^{int}$ of each layer
- 9: Derive $\mathcal{L}_{\mathcal{T}}$ and $\mathcal{L}_{\mathcal{G}}$ according to Eq. 15 and Eq. 17
- 10: Derive $\mathcal{L}_{CL}$ according to Eq. 18
- 11: $\mathcal{L}_{HCL} \leftarrow \mathcal{L}_{\mathcal{T}} + \mathcal{L}_{\mathcal{G}} + \mathcal{L}_{CL}$
- 12: Update parameters by gradient descent
- 13: **Return:** $\bar{\mathbf{H}}^{cro}, \bar{\mathbf{H}}^{int}$

weights for knowledge concepts at different levels poses a significant challenge. To address this, we devised a weight allocation networks capable of dynamically adjusting the influence of knowledge concepts across hierarchical levels.

The latent state of learners is obtained as:

$$\mathbf{u}_t \leftarrow \text{LSTM}(\mathbf{x}_t, \mathbf{u}_{t-1}^p, \theta_p), \quad \tilde{\mathbf{u}}_t \leftarrow \text{LSTM}(\mathbf{u}_t, \tilde{\mathbf{u}}_{t-1}, \theta_w), \quad (20)$$

where $\mathbf{u}_t, \tilde{\mathbf{u}}_t \in \mathbb{R}^{l \times d}$ represent the latent states, while θ_p and θ_w denote additional parameters of the LSTM. $\mathbf{x}_t$ represents the interaction context at time t . It is split as:

$$\mathbf{u}_t^k \leftarrow \text{Split}(\mathbf{u}_t, I, k), \quad \tilde{\mathbf{u}}_t^k \leftarrow \text{Split}(\tilde{\mathbf{u}}_t, I, k), \quad (21)$$

where Split denotes the splitting function, I represents the total number of hierarchical levels of the knowledge concepts, and $\mathbf{u}_t^k$ and $\tilde{\mathbf{u}}_t^k$ represent the latent states of the k th level after the splitting process.

Subsequently, the influence weights of knowledge concepts at each hierarchical level are computed as:

$$w_t^k = \frac{\exp\left(\text{ReLU}\left(\mathbf{W}_w (\tilde{\mathbf{u}}_t^k \parallel \mathbf{h}_v^{cro})^\top\right)\right)}{\sum_{v \in \mathcal{P}} \exp\left(\text{ReLU}\left(\mathbf{W}_w (\tilde{\mathbf{u}}_t^k \parallel \mathbf{h}_v^{cro})^\top\right)\right)}, \quad \phi(v) = k, \quad (22)$$

where $\mathbf{W}_w \in \mathbb{R}^{2d}$ represents the weight matrix, and w^k denotes the weight assigned to the knowledge concept at the k th hierarchical level corresponding to node v in the hierarchical path $\mathcal{P}$. The weights at different levels represent the extent to which various levels of knowledge concepts influence the learner's answer to the current question. By combining the learner's performance (correct or incorrect), this weight distribution mechanism can effectively help identify the learner's weaknesses in each knowledge level and provide clear reasoning for improvement.

Training the weight allocation networks is framed as a multi-class classification task. Optimization involves comparing the predicted results of knowledge concepts at different hierarchical levels with their ground truth. Initially, the predicted results for knowledge concepts at each hierarchical level are computed as:

$$\hat{r}_t^k = \sigma(\mathbf{u}_t^k \cdot \tilde{\mathbf{h}}_v^{cro}), \quad v \in \mathcal{P} \text{ and } \phi(v) = k. \quad (23)$$

The weights w^k are treated as a probability distribution for the multi-class classification task. It is optimization is performed as:

$$\mathcal{L}_w = -\log(w^i), \quad i = \arg \min_{k \in I} |\hat{r}^k - r|, \quad (24)$$

where $\arg \min_{k \in I} |\hat{r}^k - r|$ represents the level where the predicted value $\hat{r}^k$ is closest to the actual observed value r .

The allocation of weights must adhere to specific conditions, ensuring that predictions from different levels are combined in accordance with their respective weights, such that the results at each level correspond closely to the actual outcomes:

$$\mathcal{L}_p = -\sum_{s \in S} r \log\left(\sum_{k=1}^I w^k \hat{r}^k\right) + (1-r) \log\left(1 - \sum_{k=1}^I w^k \hat{r}^k\right). \quad (25)$$

Finally, the weight allocation networks are optimized as:

$$\mathcal{L} = \mathcal{L}_w + \mathcal{L}_p. \quad (26)$$

Algorithm 2 Training of weight allocation networks

Input: $\bar{\mathbf{H}}^{cro}$, $\mathbf{x}_t$
Output: The weight of w^i , the prediction of $\hat{r}^i$

- 1: **while** KT performance is not optimal **do**
- 2: Derive u_t and $\tilde{u}_t$ according to Eq. 20
- 3: Derive u_t^k and $\tilde{u}_t^k$ according to Eq. 21
- 4: Derive w_t^k according to Eq. 22 using u_t^k and h_v^{cro}
- 5: $\hat{r}_t^k \leftarrow \sigma(u_t^{pk} \cdot h_v^{cro}(i))$
- 6: Derive L_w and L_p according to Eq. 24 and Eq. 25
- 7: $\mathcal{L} \leftarrow \mathcal{L}_p + \mathcal{L}_w$
- 8: Update parameters by gradient descent
- 9: **end while**
- 10: **Return:** w^i , $\hat{r}^i$

4.6. Optimization

In this work, we employed a phased training strategy. Initially, the embeddings of KHT and KAG are optimized using graph reconstruction and cross-graph hierarchical contrastive learning methods. These optimized embeddings are then incorporated into the weight allocation networks, which are jointly optimized with both the weight allocation loss $\mathcal{L}_w$ and the prediction loss $\mathcal{L}_p$. The entire process is optimized using the Adam optimizer and the algorithm is shown in Algorithms 1 and 2.

4.7. Complexity analysis

The complexity of the proposed HKT model consists of two main components: (1) the computation of knowledge concept representations via KHT and KAG, and (2) the performance prediction through the weight allocation module. Specifically, for the knowledge concept representation part, the complexity is $O(l \cdot k \cdot |G|)$, where l represents the number of encoding layers, k is the number of negative samples sampled in hierarchical contrastive learning, and $|G|$ is the number of nodes in the constructed KHT or KAG. Since both have the same number of nodes, no separate distinction is made. For the weight allocation networks module, the complexity is $O(T)$, where T is the length of the historical exercise records. Therefore, the overall time complexity of the HKT model is $O(l \cdot k \cdot |G|) + O(T)$, which is approximately $O(n)$, significantly lower than $O(n^2)$.

5. Experiments

In this section, we evaluate our model's performance on the KT task by comparing it with the baseline models on public datasets. Experiments are conducted to address the following questions:

1. **Q1:** Does our approach demonstrate a clear advantage over baseline methods?
2. **Q2:** Are the components of HKT effective, and how do they impact the model's performance?
3. **Q3:** Is HKT sensitive to hyperparameters, and how do these hyperparameters influence its performance?
4. **Q4:** Is hierarchical modeling effective for the KT task, and does multi-hierarchical knowledge feature joint prediction outperform single-hierarchical predictions?
5. **Q5:** How does hierarchical contrastive learning of knowledge concepts at different stages impact subsequent KT tasks?

5.1. Experimental setup

5.1.1. Dataset and evaluate metrics

We conduct experiments on three publicly available datasets:

- **ASSIST2017**¹: Collected from the ASSISTments platform developed by MIT, the dataset records learners' interactions in mathematics, including anonymized learner details, problem data, responses, solving times, and hint usage. It is widely used in educational research such as data mining, knowledge tracing, and personalized learning.
- **Eedi**²: From the Eedi platform, the dataset contains anonymized learner profiles, question details, response logs, solving times, and hint usage across multiple subjects. It is frequently applied in studies on data mining, knowledge tracing, and adaptive teaching.

¹ <https://sites.google.com/view/assistmentsdatamining/dataset>.

² <https://eedi.com/projects/neurips-education-challenge>.

Table 2
The statistics of datasets.

Dataset	ASSIST2017	Eedi	Junyi
#student	1.7k	118.9k	247.6k
#question	3.2k	26.0k	721
#response	0.9 m	15.9 m	25.9 m
#hierarchy	102 (L1)	7 (L1)	8 (L1)
	3.2k (L2)	108 (L2)	39 (L2)
		264 (L3)	721 (L3)
		26.0k (L4)	

- **Junyi³**: Sourced from the Junyi platform, the dataset includes anonymized learner data, question metadata, responses, solving times, and hint usage in various subjects. It supports research in educational data mining, knowledge tracing, and personalized instruction.

Each learner's response sequence is split into subsequences of 100 interactions, with shorter sequences padded for uniformity. Statistics of the processed datasets can be found in Table 2. As predicting learner responses constitutes a binary classification task, we utilize Area Under the Curve (AUC), Accuracy (ACC), and Root Mean Square Error (RMSE) as evaluation metrics. For more details regarding the above metrics, please refer to Cui, Yu et al. (2024).

5.1.2. Baseline methods

According to the modeling approach of knowledge concept structures, we compare our method with three types of KT models: pointwise knowledge tracing, implicit knowledge tracing, and explicit knowledge tracing.

Pointwise Knowledge Tracing (POI-KT): A class of KT models based on the hidden markov model, where each knowledge concept is treated as an independent variable:

- **BKT**: Based on Bayesian networks, BKT (Corbett & Anderson, 1994) models learners' knowledge states as binary variables for each knowledge concept. It is one of the earliest and most fundamental methods in the field of knowledge tracing.

Implicit Knowledge Tracing (IMP-KT): A class of KT models that implicitly considers the underlying relationships between knowledge concepts through the hidden layers of the model:

- **DKT**: Based on recurrent neural networks (RNN), DKT (Piech et al., 2015) employs deep learning to track learners' dynamically changing knowledge states. It is a pioneering work that introduced deep learning into knowledge tracing.
- **DKVMN**: Based on dynamic memory mechanisms, DKVMN (Zhang, Shi, King, & Yeung, 2017) utilizes a dynamic key-value memory network to record the interaction history between learners and questions, predicting learners' performance on new questions.
- **DKT+**: Based on the forgetting curve, DKT+ (Yeung & Yeung, 2018) incorporates forgetting information to better capture learners' knowledge forgetting patterns, thereby improving prediction accuracy.
- **SAKT**: Based on the self-attention mechanism, SAKT (Xu, Ruan, Korpeoglu, Kumar, & Achan, 2019) focuses on the relations between learners' past responses and current questions, predicting learners' performance on new questions.
- **AKT**: Based on the attention mechanism, AKT (Ghosh et al., 2020) captures global dependencies and incorporates temporal interval information to model long-range dependencies, thereby improving the performance of KT.
- **KQN**: Based on the query mechanism, KQN (Lee & Yeung, 2019) treats questions as queries and dynamically generates learners' knowledge states, enabling flexible modeling of knowledge states.
- **LPKT**: Based on learning process information, LPKT (Shen et al., 2021) uses data such as timestamps, attempt counts, and response durations to accurately track learners' learning states.
- **SimpleKT**: Based on problem variation modeling, SimpleKT (Liu, Liu, Chen, Huang, & Luo, 2023) explicitly captures individual differences between problems, making it a simple yet useful baseline model.
- **DTransformer**: Based on the Diagnostic Transformer, DTransformer (Yin et al., 2023) builds a hierarchical framework from the question level to the knowledge level, explicitly diagnosing learners' knowledge proficiency for each question and incorporates contrastive learning to enhance performance.
- **StableKT**: Based on a linear-biased attention mechanism, StableKT (Li, Bai, Guo, Liu et al., 2024) addresses the sequence length generalization problem through distance-proportional attention penalties, effectively maintaining stable predictions across interaction sequences of varying lengths while enhancing performance.
- **ReKT**: Based on human teachers' decision-making processes and multi-level cognitive development theory, ReKT (Shen et al., 2024) comprehensively captures students' dynamic knowledge states through multi-perspective knowledge state modeling and enhances KT performance by designing a lightweight FRU framework to simulate cognitive patterns.

³ <https://pslcdatashop.web.cmu.edu/Files?datasetId=1198>.

Table 3

Results of the main experiments. The best results between HKT and the baselines are highlighted in bold and the second-best results are underlined.

Type	Metric	ASSIST2017			Eedi			Junyi		
		AUC	ACC	RMSE	AUC	ACC	RMSE	AUC	ACC	RMSE
POI-KT	BKT	0.6368	0.6463	0.4700	0.6226	0.6513	0.4683	0.6206	0.9609	0.4688
IMP-KT	DKT	0.7430	0.6958	0.4464	0.7818	0.7299	0.4317	0.7947	0.8509	0.3573
	DKVMN	0.7424	0.6992	0.4430	0.7941	0.7394	0.4167	0.7953	0.8514	0.3344
	DKT+	0.7539	0.7035	0.4469	0.7818	0.7310	0.4379	0.7894	0.8484	0.3632
	SAKT	0.6382	0.6544	0.4712	0.7470	0.7047	0.4413	0.7392	0.8367	0.3523
	AKT	0.7586	0.7090	0.4359	<u>0.7977</u>	<u>0.7417</u>	<u>0.4152</u>	0.7971	0.8520	0.3340
	KQN	0.7281	0.6848	0.4483	0.7974	0.7396	0.4169	0.7895	0.8501	0.3363
	LPKT	0.7643	<u>0.7152</u>	0.4332	0.7003	0.6796	0.4533	0.7889	0.8437	0.3445
	SimpleKT	<u>0.7679</u>	0.7151	<u>0.4322</u>	0.7972	0.7408	0.4156	0.7974	0.8521	0.3338
	Dtrans	0.7624	0.7134	0.4335	0.7909	0.7369	0.4223	0.7879	0.8501	0.3390
	ExtraKT	0.7649	0.7154	0.4317	0.7663	0.7213	0.4289	0.7960	0.8517	0.3351
	StableKT	0.7614	0.7135	0.4332	0.7940	0.7338	0.4188	0.7952	0.7347	0.4182
	ReKT	0.7675	0.7186	0.4306	0.7884	0.7365	0.4203	0.7972	0.8538	0.3323
EXP-KT	GKT	0.7596	0.7436	0.4150	0.7832	0.7309	0.4219	0.7937	0.8511	0.3352
	GRKT	0.7612	0.7086	0.4373	0.7469	0.7064	0.4365	<u>0.7983</u>	<u>0.8524</u>	<u>0.3336</u>
HIE-KT	HKT	0.7703	0.7165	0.4317	0.8010	0.7435	0.4140	0.7995	0.8541	0.3320
	improv.	0.31%	0.18%	0.11%	0.41%	0.24%	0.14%	0.41%	0.19%	0.48%

- **ExtraKT**: By leveraging length extrapolation mechanisms, optimized attention bias, and fine-grained interaction representations, ExtraKT (Li, Bai, Guo, Zheng et al., 2024) addresses the dependency of traditional KT models on fixed window lengths, enhancing adaptability to variable-length sequences and prediction accuracy in real-world educational scenarios.

Explicit Knowledge Tracing (EXP-KT): A class of KT models that explicitly represents the relationships between knowledge concepts through a graphical model:

- **GKT**: Based on graph neural networks, GKT (Nakagawa et al., 2019) learns graph relations between knowledge concepts and questions to capture higher-order information, improving KT performance.
- **GRKT**: Based on graph structure modeling, GRKT (Cui, Qian, Jiang and Zhang, 2024) captures mutual influences between knowledge concepts through three-stage modeling, enhancing the rationality and performance of knowledge tracing.

The parameters and hyperparameters for each baseline model are configured following the guidelines and methods outlined in their original research papers.

5.1.3. Implementation details

The model is implemented using the PyTorch framework, and the Adam optimizer (Kingma & Ba, 2014) is employed for optimization. During the training process, hyperparameter tuning is conducted to determine the optimal configuration. In the experiments, the embedding dimension d is selected from {32, 64, 128, 256}, the learning rate is chosen from {0.0005, 0.001, 0.002}, the weight decay coefficient is selected from {1e-2, 1e-3, 1e-4}, the dropout rate is chosen from {0.1, 0.2, 0.3}, the number of encoding layers is selected from {1, 2, 3, 4}, the temperature coefficient τ is chosen from {0.1, 0.5, 1, 2}, and the scaling factor γ is selected from {1, 2, 4, 8}. For each dataset, it is split into training and testing sets in the ratio of 8:2. Ultimately, the learning rate is set to 0.001, weight decay is set to 1e-4, the dropout rate is set to 0.1, and the embedding dimension is set to 128. The number of encoding layers, temperature coefficient τ , and scaling factor γ are determined based on the specific characteristics of the dataset, and a more detailed discussion of these aspects is provided in Section 5.4. The sequence length for all learners is uniformly processed to 100; for students with sequence lengths less than 100, padding with zeros is applied, while sequences longer than 100 are sliced. All experiments are conducted on a PC server equipped with an Intel® Core™ i7-14700K CPU and an NVIDIA GeForce RTX 4090D GPU. The models are implemented using the PyTorch 2.0.0 framework.

As predicting learner responses constitutes a binary classification task, we evaluate the effectiveness of HKT using three widely applied evaluation metrics: area under the ROC curve (AUC), accuracy (ACC), and root mean square error (RMSE). AUC measures the model's ability to distinguish between the samples correctly, ACC assesses the overall prediction accuracy, and RMSE evaluates the variance of predicted values from actual values. Higher AUC and ACC values indicate better performance, while lower RMSE values are superior. For more details regarding the above metrics, please refer to Cui, Yu et al. (2024).

5.2. Overall performance (Q1)

The experimental results of HKT in comparison with other baseline models are presented in Table 3, which can be summarized as follows:

1. HKT consistently outperformed baseline methods across all datasets, with AUC increasing by 0.31% to 0.41%, ACC by 0.18% to 0.24%, and RMSE decreasing by 0.11% to 0.48%, Which shows the effectiveness of hierarchical knowledge-level modeling.

Table 4
Results of the ablation experiments.

Dataset	ASSIST2017			Eedi			Junyi		
Metric	AUC	ACC	RMSE	AUC	ACC	RMSE	AUC	ACC	RMSE
HKT	0.7703	0.7165	0.4317	0.8010	0.7435	0.4140	0.7981	0.8529	0.3331
WO-KHT	0.7665	0.714	0.4321	0.7961	0.741	0.416	0.7936	0.8504	0.3354
drop	0.49%	0.35%	0.09%	0.61%	0.33%	0.48%	0.56%	0.29%	0.69%
WO-KAG	0.7539	0.7052	0.438	0.7965	0.7413	0.4156	0.7943	0.8502	0.3356
drop	2.12%	1.57%	1.45%	0.56%	0.29%	0.38%	0.47%	0.31%	0.75%
WO-HCL	0.7608	0.7104	0.4344	0.7923	0.7378	0.4177	0.7958	0.8509	0.3349
drop	1.23%	0.85%	0.62%	1.08%	0.76%	0.89%	0.28%	0.23%	0.54%

- IMP-KT and EXP-KT significantly outperform POI-KT, underscoring the necessity of modeling correlations between knowledge concepts in KT tasks. However, no significant difference was observed between IMP-KT and EXP-KT, possibly because both methods overlook the hierarchical relations within knowledge. The proposed hierarchical KT effectively addresses this shortcoming.
- The advantage of HKT on the Junyi dataset is relatively limited, possibly due to the smaller number of questions, which may have affected the model's performance to some extent.

Ultimately, our model achieved the best results across multiple evaluation metrics. Even on the Assistment dataset, which has a sparse knowledge hierarchy structure, HKT performed exceptionally well, indicating its robustness and applicability to datasets with less-defined knowledge structures.

5.3. Ablation study (Q2)

We evaluated the contribution of each component to the overall model performance by conducting the following ablation studies:

- WO-KHT**: Removes the Knowledge Hierarchy Tree, generating knowledge concept embeddings without modeling vertical hierarchical relations.
- WO-KAG**: Removes the Knowledge Association Graph, generating knowledge concept embeddings without modeling intra-layer associations.
- WO-HCL**: Removes the Hierarchical Contrastive Learning Module, generating knowledge concept embeddings without employing hierarchical contrastive learning.

As shown in Table 4, we find:

- Removing the KHT reduced the AUC by 0.35%, 0.61%, and 0.56% across the three datasets, highlighting its positive impact on capturing vertical knowledge relations, especially in the more complex Eedi and Junyi datasets.
- Removing the KAG caused the largest AUC drop of 2.12% in the ASSIST2017 dataset, showing that intra-layer relations are more crucial in datasets with simpler hierarchical structures.
- Removing the HCL led to performance drops in all datasets, confirming its effectiveness in improving overall model performance. The performance drop on the Junyi dataset is smaller compared to the other two datasets. It is due to the dataset which has fewer knowledge concepts, especially a scarcity of knowledge concepts at the lower leaf nodes. As a result, the sample size for hierarchical contrastive learning is relatively small, leading to a more limited performance decline after removing the HCL module.

Differences in knowledge hierarchies likely drive the varying impact of different components. The ASSIST2017 dataset has fewer hierarchical levels. Removing the KHT, which models relations across levels, causes only a minor performance drop. In contrast, the Eedi dataset features more complex hierarchical structures. In this case, removing KHT results in a greater performance drop compared to component B. These findings indicate that richer knowledge hierarchies enhance the effectiveness of KHT. Meanwhile, contrastive learning proves to be more effective in datasets with simpler hierarchical structures.

5.4. Parameter sensitivity analysis (Q3)

We conducted experiments on key hyperparameters to analyze their impact on model performance. The results are presented in Fig. 3.

- In the ASSIST2017 and Junyi datasets, the AUC metric performs best with two encoding layers, while in the Eedi dataset, both AUC and ACC reach their peak with three encoding layers. This result aligns with the hierarchical structure of the datasets. Since the ASSIST2017 and Junyi datasets have fewer levels, fewer encoding layers are required. In contrast, the Eedi dataset has more levels, necessitating more encoding layers to better capture the complex hierarchical relationships.

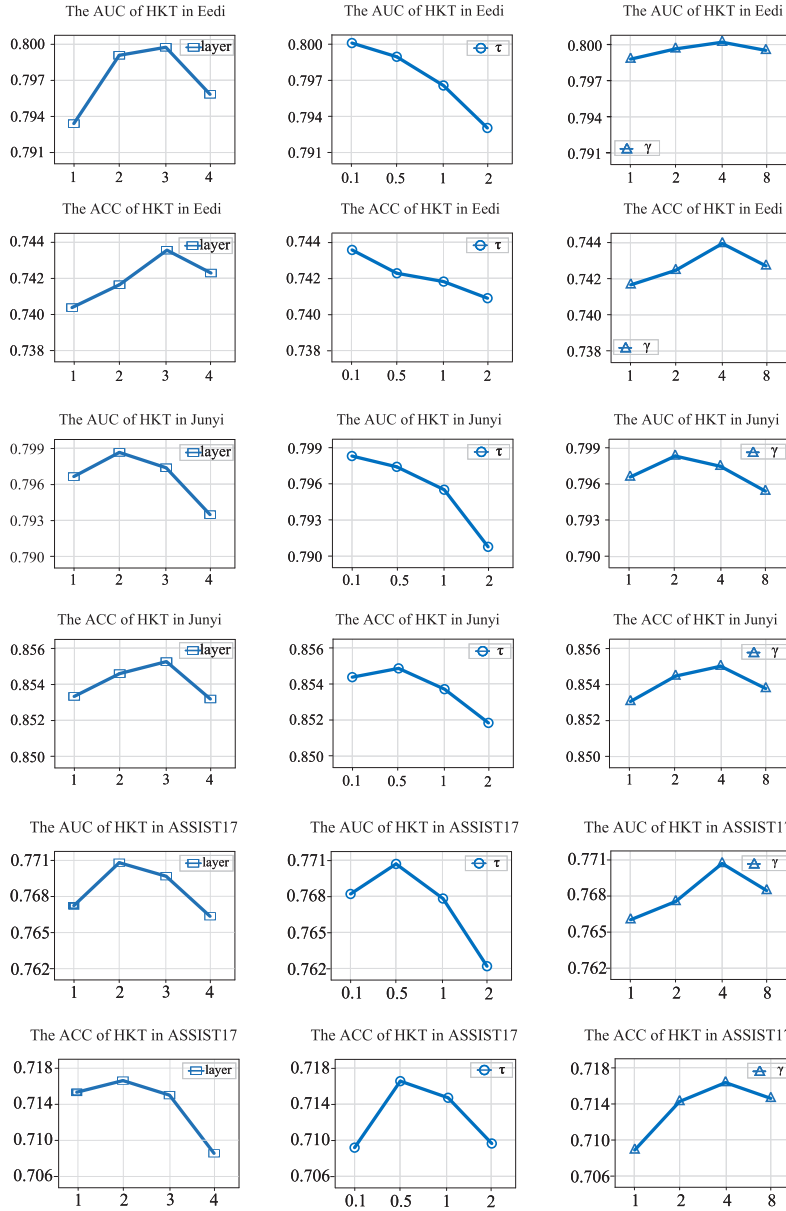


Fig. 3. Results of hyper-parameter sensitivity analysis. The layer represents the number of encoding layers, the temperature coefficient τ is used to control the score scale of positive and negative sample pairs, and the scaling factor γ is used to adjust the hierarchical contrastive learning exponent.

2. A lower temperature coefficient τ helps to improve graph reconstruction performance. By enhancing the embedding representations of knowledge concepts, it further boosts the model's performance in prediction tasks, thereby improving overall model effectiveness.
3. The scaling factor γ in contrastive learning exhibits low sensitivity. In all datasets, the change in AUC values is relatively small, and the performance differences across different γ values are stable. Typically, when γ is set to 2 or 4, the model achieves its best performance.

The optimal number of encoding layers corresponds to the complexity of the hierarchical structure of the dataset. Through effective hyperparameter tuning, the HKT model consistently achieves stable and superior performance across datasets, demonstrating its robustness and adaptability.

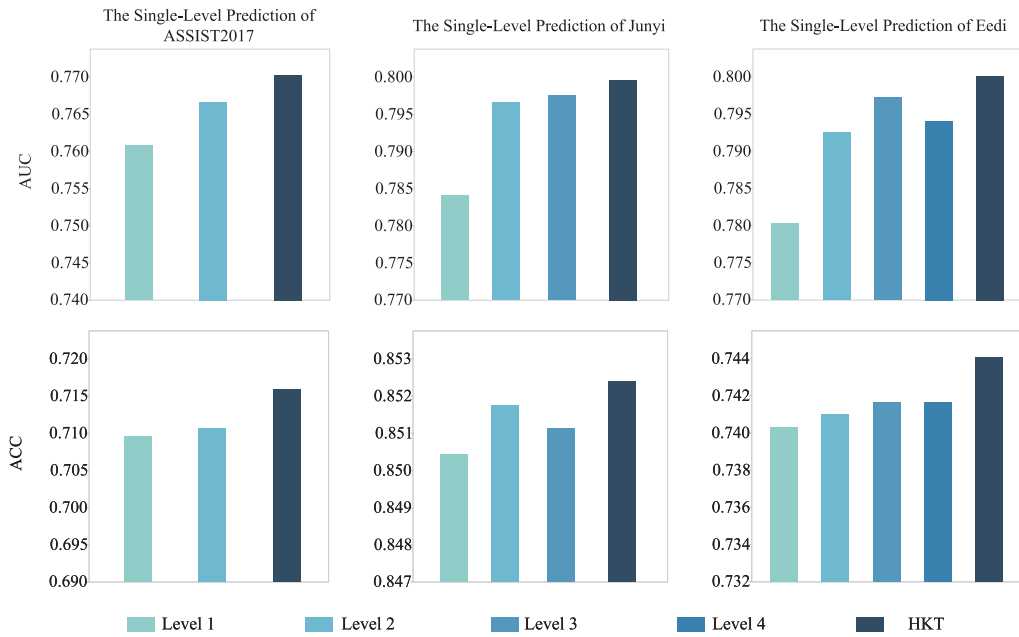


Fig. 4. The results of single-level prediction. A comparison between the performance of prediction tasks using knowledge concepts from a single level and the comprehensive prediction integrating knowledge concepts across multiple levels.

5.5. Single-level knowledge-driven prediction experiment (Q4)

The effectiveness of multi-level weight is tested through independent KT experiments using knowledge concepts from different hierarchical levels, as shown in Fig. 4:

1. Employing a single level for prediction leads to performance declines across all datasets, showing that relying on one level is insufficient for accurately predicting learners' final responses.
2. While the overall trend is consistent, AUC performance varies across datasets at different levels. For instance, Eedi and Junyi exhibit greater improvements at deeper levels. It is likely due to their larger number of knowledge concepts and more enriched hierarchical structures.

Combining information from all levels, rather than a single level, produces the most accurate predictions across all datasets. This confirms the effectiveness of our multi-level approach, showing that hierarchical knowledge structures better support complex reasoning and problem-solving. It aligns with cognitive architecture theories and illustrates the potential of hierarchical frameworks to enhance the capability of knowledge tracing tasks.

5.6. Effect of embedding training on performance

The ablation experiments reveal that the KHT and the KAG significantly influence the experimental results. To further quantify the impact of graph-based training embeddings on model performance, we performed t-SNE visualizations of question embeddings at different training stages. We selected five types of question nodes representing distinct knowledge concepts and tracked their embeddings throughout the training process in ASSIST2017. The results are presented in Fig. 5:

1. Through multiple training iterations, the initially random distribution of question nodes gradually converges into tightly packed clusters.
2. Different types of question nodes move apart, with increasingly distinct boundaries, showing the model's ability to capture conceptual differences.
3. Untrained model embeddings perform poorly in predictions, but our hierarchical modeling and contrastive learning greatly improve results, highlighting the approach's effectiveness.

The visualization results clearly illustrate the structured evolution of question embeddings during training. As training progresses, contrastive learning and hierarchical modeling gradually take effect: similar questions cluster tightly in low-dimensional space, while boundaries between categories become sharper. Correspondingly, the embeddings show steady improvements in prediction tasks. This demonstrates the model's ability to capture knowledge relations and differences between questions, while validating the effectiveness of the multi-level contrastive learning strategy in optimizing embeddings and enhancing semantic expressiveness.

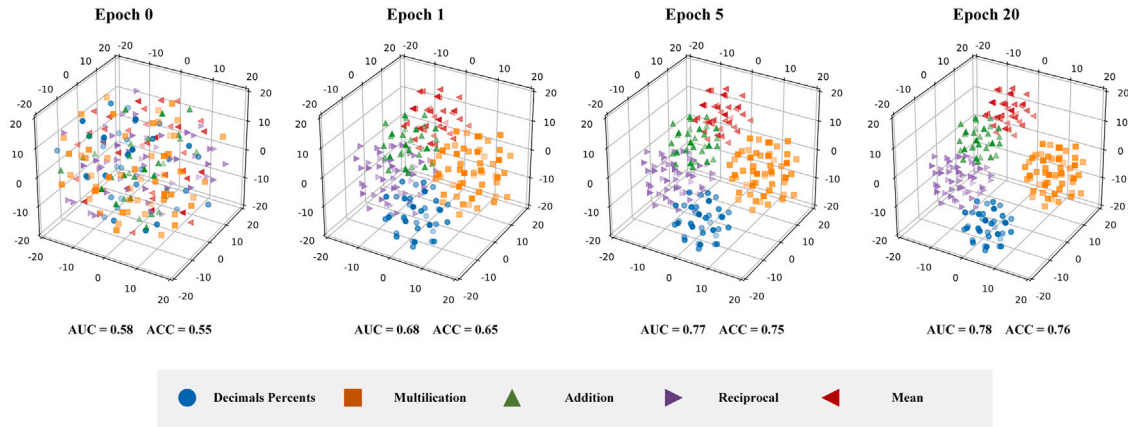


Fig. 5. The visualization of question embedding. Nodes of the same color represent question nodes associated with the same knowledge concept. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 5

Model training efficiency and resource consumption.

Model	Parameters (M)	Epoch time (s)	Throughput (samples/s)	Inference time/Step (ms)	GPU memory (GB)
BKT	0.01	8.67	940	0.31	0.1
DKT	1.03	2.66	3100	0.02	1.1
DKT+	1.03	6.85	1200	0.05	2
DKVMN	0.49	4.21	2100	0.06	0.3
KQN	1.40	2.33	3500	0.02	0.4
GKT	0.39	61.09	133	3.54	19
SAKT	0.25	2.72	2800	0.02	0.3
AKT	4.94	1.05	7745	0.16	3.3
ExtraKT	4.38	5.96	1400	0.12	3.3
Dtrans	0.35	9.96	850	0.35	8.7
LPKT	0.66	13.73	600	0.73	3.6
ReKT	0.77	23.53	350	1.34	0.3
SimpleKT	2.02	3.02	2800	0.03	0.8
StableKT	2.78	4.32	1900	0.07	2.1
Mean	1.50	10.72	2100	0.49	3.2
HKT	0.52	2.80	2900	0.02	1.3

5.7. Model complexity and deployment feasibility analysis

We performed a unified parameter-based evaluation of the training efficiency and resource consumption of various models on the ASSIST2017 dataset, with the results shown in Table 5. Overall, our method demonstrates a balanced performance in terms of training time, inference speed, and computational resource consumption, particularly excelling in single-step inference speed, where it outperforms most deep learning-based KT methods. It is important to note that the GKT model involves extensive batch matrix operations and graph convolutional layers, processed step-by-step with only one time step of data being handled at a time. This results in lower efficiency compared to other models. However, excluding the GKT model, our method still shows excellent computational efficiency when compared to several baseline models.

From the perspective of deployment feasibility, our method can be flexibly deployed on current mainstream ITS platforms. Although our method consists of multiple modules, each one is not complex and follows a clear separation between the front-end and back-end. Specifically, our approach is composed of two main stages: Graph Embedding and Weight Allocation Networks. These two components are designed to achieve high cohesion and low coupling. The graph embedding stage only needs to learn the features of knowledge concepts and provide them to the weight allocation network, which drives the entire model's operation. In actual deployment, switching to different graph embedding schemes or subsequent backbone networks will not affect the overall functionality of the model, as long as the interfaces between them remain consistent. Moreover, our knowledge concept representation method, based on the KHT and KAG structures, employs an incremental learning approach, allowing for offline learning with new data and dynamic online deployment. This means our method can be easily adapted to mainstream ITS platforms and quickly put into operation.

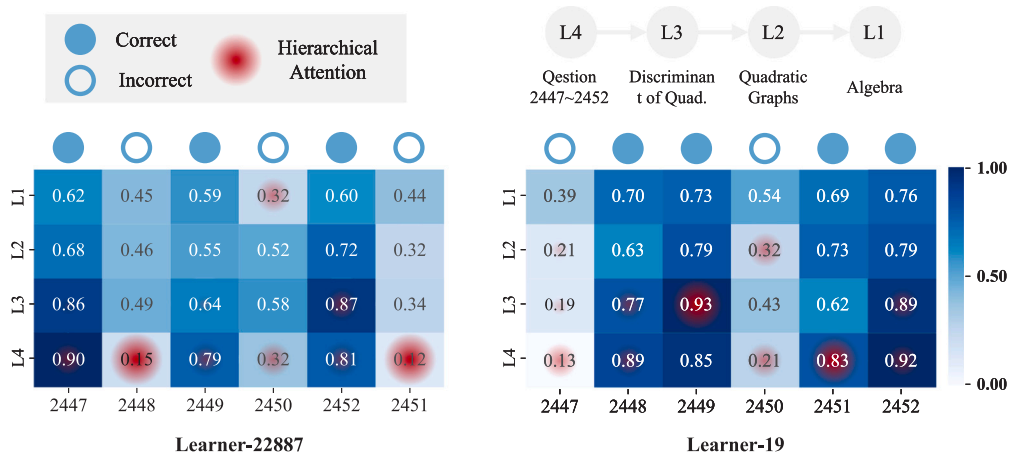


Fig. 6. The illustration of a case study involving two learners from the Eedi dataset. The numbers within the cells indicate the prediction results based on the knowledge concepts at each hierarchical level. The red shading represents the weights assigned by the weight allocation model, with the intensity of the shading reflecting the magnitude of the assigned weight.

5.8. Case study

A clearer understanding of the multi-level prediction process is provided by analyzing the response data of two learners from the Eedi dataset. They have similar response sequences but different outcomes. A detailed comparison of predictions across various hierarchical knowledge levels is presented in Fig. 6. Comparing the predictions to the actual responses, our weight allocation model reliably determines the level that best matches each learner's performance. For example, when answering question 2449, learner 22887 is mainly influenced by the L4 level, while learner 19 relies more on the L3 level. Interestingly, learner 22887 answered the previous question incorrectly, whereas learner 19 answered it correctly. Despite the prior response differences, the model accurately pinpoints the most relevant level and assigns it the highest weight. It shows that the model's ability to effectively evaluate the influence of different levels, even in complex scenarios.

Meanwhile, the weight allocation provides educators with interpretable instructional assistance information. When learners answer questions, the weight allocation networks can offer reference weights for different knowledge levels involved in each question. Through this information, teachers can assess learners' mastery of various knowledge levels. If a learner answers correctly, the teacher can analyze their knowledge mastery based on the weight allocation; if the answer is incorrect, the teacher can identify which knowledge level the learner struggles with and infer the potential cause of the error. For example, when learner 22887 answered question 2448, the weight allocation indicated that the L4 level knowledge concept might be the area of insufficient mastery, leading to the incorrect response. Thus, weight allocation helps teachers precisely locate learners' weaknesses, enabling targeted guidance and remediation.

Additionally, analyzing the prediction results at different levels, it was observed that the L1 level exhibits relatively stable variations compared to other levels. We speculate that it is related to the hierarchical structure in the Eedi. The L1 level contains only 7 knowledge concepts, which limits the range of embedding options available at this level. As a result, the limited embedding options at this level likely lead to smoother in prediction outcomes during single-level predictions.

6. Conclusion and future work

We present the HKT, a hierarchical and interpretable knowledge tracing model that effectively captures both vertical and horizontal dependencies between knowledge concepts through the integration of a KHT and a KAG. In addition, we have designed a weight allocation networks that dynamically and accurately assign weights to knowledge at various hierarchical levels, enabling a comprehensive prediction of learner performance. The experimental results demonstrate that HKT consistently outperforms existing pointwise, implicit, and explicit knowledge tracing models across multiple publicly available datasets. Further ablation studies and hyperparameter analysis confirm the importance and robustness of each component within the model. Meanwhile, HKT also demonstrates good interpretability by explicitly modeling knowledge concepts through KHT and KAG, offering an interpretable and structured knowledge framework that aligns with human cognition. It then employs a dynamic weight allocation mechanism to provide educators with a clear and interpretable basis for precisely identifying the learners' weak knowledge levels. This work introduces a novel approach to accurately model and predict learners' knowledge states, offering valuable insights for the development of personalized education systems.

In the future, we aim to explore the potential of self-supervised learning to discover knowledge structures. On the one hand, manually annotated knowledge structures require significant human resources and are inefficient. On the other hand, manually annotated knowledge structures may not necessarily align well with deep learning models. We look forward to investigating whether there are differences between the knowledge structures learned by deep learning models and those annotated by humans.

CRediT authorship contribution statement

Qing Li: Supervision, Project administration, Data curation, Conceptualization. **Zhijun Huang:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation. **Jianwen Sun:** Writing – review & editing, Supervision, Software, Project administration. **Xin Yuan:** Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Shengyingjie Liu:** Writing – review & editing, Visualization, Supervision, Software, Resources, Methodology, Conceptualization. **Zhonghua Yan:** Supervision, Resources, Methodology.

Informed consent

All authors agree to submit this edition and declare that no part of this manuscript has been published or submitted elsewhere. We thank you for your consideration of our manuscript and look forward to receiving comments from the reviewers as soon as possible.

Funding

This work was supported by the General Project of the National Social Science Fund of China (grant number BCA220210).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data sharing not applicable to this article as no datasets were generated or analyzed during the current study.

References

- Abdelrahman, G., Wang, Q., & Nunes, B. (2023). Knowledge tracing: A survey. *ACM Computing Surveys*, (11).
- Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, (8).
- Bulut, O., Shin, J., Yildirim-Erbasli, S. N., Gorgun, G., & Pardos, Z. A. (2023). An introduction to Bayesian knowledge tracing with pybkt. *Psych.*
- Casolino, G., Grilli, L., Limone, P., Santoro, D., Schicchi, D., et al. (2021). Deep learning for knowledge tracing in learning analytics: an overview. *TeleXbe*.
- Chen, F., Wang, Y.-C., Wang, B., & Kuo, C.-C. J. (2020). Graph representation learning: a survey. *APSIPA Transactions on Signal and Information Processing*, 9.
- Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*.
- Cui, J., Qian, H., Jiang, B., & Zhang, W. (2024). Leveraging pedagogical theories to understand student learning process with graph-based reasonable knowledge tracing. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*.
- Cui, J., Yu, M., Jiang, B., Zhou, A., Wang, J., & Zhang, W. (2024). Interpretable knowledge tracing via response influence-based counterfactual reasoning. In *2024 IEEE 40th international conference on data engineering*. IEEE.
- Falakmasir, M., Pardos, Z., Gordon, G. J., & Brusilovsky, P. (2013). A spectral learning approach to knowledge tracing.
- Ghosh, A., Heffernan, N., & Lan, A. S. (2020). Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*.
- Grover, A., & Leskovec, J. (2016). Node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*.
- Gutmann, M., & Hyvärinen, A. (2010). Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings.
- Hamilton, W. L. (2020). *Graph representation learning*. Morgan & Claypool Publishers.
- Huang, Z., Li, K., Jiang, Y., Jia, Z., Lv, L., & Ma, Y. (2024). Graph relearn network: Reducing performance variance and improving prediction accuracy of graph neural networks. *Knowledge-Based Systems*.
- Jiang, B., Wei, Y., Zhang, T., & Zhang, W. (2024). Improving the performance and explainability of knowledge tracing via Markov blanket. *Information Processing & Management*, (3).
- Khosrhafta, S., & An, A. (2024). A survey on graph representation learning methods. *ACM Transactions on Intelligent Systems and Technology*, (1).
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
- Lee, J., & Yeung, D.-Y. (2019). Knowledge query network for knowledge tracing: How knowledge interacts with skills. In *Proceedings of the 9th international conference on learning analytics & knowledge*.
- Li, X., Bai, Y., Guo, T., Liu, Z., Huang, Y., Zhao, X., et al. (2024). Enhancing length generalization for attention based knowledge tracing models with linear biases. In *Proceedings of the thirty-third international joint conference on artificial intelligence (IJCAI-24)*.
- Li, X., Bai, Y., Guo, T., Zheng, Y., Hou, M., Zhan, B., et al. (2024). Extending context window of attention based knowledge tracing models via length extrapolation. In *ECAI 2024*. IOS Press.
- Li, Z., Jian, Y., Xue, Z., Zheng, Y., Zhang, M., Zhang, Y., et al. (2024). Text-enhanced knowledge graph representation learning with local structure. *Information Processing & Management*, (5).
- Liu, F., Hu, X., Bu, C., & Yu, K. (2022). Fuzzy Bayesian knowledge tracing. *IEEE Transactions on Fuzzy Systems*.
- Liu, Z., Liu, Q., Chen, J., Huang, S., & Luo, W. (2023). simpleKT: a simple but tough-to-beat baseline for knowledge tracing. arXiv preprint arXiv:2302.06881.
- Loomis, L. H. (2013). *Introduction to abstract harmonic analysis*. Courier Corporation.
- Nakagawa, H., Iwasawa, Y., & Matsuo, Y. (2019). Graph-based knowledge tracing: modeling student proficiency using graph neural network. In *IEEE/WIC/aCM international conference on web intelligence*.
- Oord, A. v. d., Li, Y., & Vinyals, O. (2018). Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748.

- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining*.
- Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L. J., et al. (2015). Deep knowledge tracing. *Advances in Neural Information Processing Systems*.
- Qian, L., Zheng, K., Wang, L., & Li, S. (2024). Student state-aware knowledge tracing based on attention mechanism: A cognitive theory view. *Pattern Recognition Letters*.
- Shen, S., Liu, Q., Chen, E., Huang, Z., Huang, W., Yin, Y., et al. (2021). Learning process-consistent knowledge tracing. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*.
- Shen, X., Yu, F., Liu, Y., Liang, R., Wan, Q., Yang, K., et al. (2024). Revisiting knowledge tracing: A simple and powerful model. In *Proceedings of the 32nd ACM international conference on multimedia*.
- Sweller, J., Van Merriënboer, J. J., & Paas, F. G. (1998). Cognitive architecture and instructional design. *Educational Psychology Review*.
- Tong, S., Liu, Q., Huang, W., Hunag, Z., Chen, E., Liu, C., et al. (2020). Structure-based knowledge tracing: An influence propagation view. In *2020 IEEE international conference on data mining*. IEEE.
- Veličković, P., Fedus, W., Hamilton, W. L., Liò, P., Bengio, Y., & Hjelm, R. D. (2018). Deep graph infomax. arXiv preprint arXiv:1809.10341.
- Wang, F., Huang, Z., Liu, Q., Chen, E., Yin, Y., Ma, J., et al. (2023). Dynamic cognitive diagnosis: An educational priors-enhanced deep knowledge tracing perspective. *IEEE Transactions on Learning Technologies*.
- Wang, Y., Lai, Y., & Huang, X. (2024). Innovations in online learning analytics: A review of recent research and emerging trends. *IEEE Access*, <http://dx.doi.org/10.1109/ACCESS.2024.3493621>.
- Wang, W., Ma, H., Zhao, Y., & Li, Z. (2024). Pre-training question embeddings for improving knowledge tracing with self-supervised bi-graph co-contrastive learning. *ACM Transactions on Knowledge Discovery from Data*, (4).
- Wang, C., & Sahebi, S. S. (2023). Continuous personalized knowledge tracing: Modeling long-term learning in online environments. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*.
- Waswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A., et al. (2017). Attention is all you need. In *NIPS*.
- Xu, D., Ruan, C., Korpeoglu, E., Kumar, S., & Achan, K. (2019). Self-attention with functional time representation learning. *Advances in Neural Information Processing Systems*.
- Yang, T., Yu, X., Ma, N., Zhang, Y., & Li, H. (2022). Deep representation-based transfer learning for deep neural networks. *Knowledge-Based Systems*.
- Yeung, C.-K., & Yeung, D.-Y. (2018). Addressing two problems in deep knowledge tracing via prediction-consistent regularization. In *Proceedings of the fifth annual ACM conference on learning at scale*.
- Yin, Y., Dai, L., Huang, Z., Shen, S., Wang, F., Liu, Q., et al. (2023). Tracing knowledge instead of patterns: Stable knowledge tracing with diagnostic transformer. In *Proceedings of the ACM web conference 2023*.
- You, Y., Chen, T., Sui, Y., Chen, T., Wang, Z., & Shen, Y. (2020). Graph contrastive learning with augmentations. *Advances in Neural Information Processing Systems*.
- Zaremba, W. (2014). Recurrent neural network regularization. arXiv preprint arXiv:1409.2329.
- Zhang, P., Niu, Z., Ma, R., & Zhang, F. (2025). Multi-view graph contrastive representation learning for bundle recommendation. *Information Processing & Management*, (1).
- Zhang, J., Shi, X., King, I., & Yeung, D.-Y. (2017). Dynamic key-value memory networks for knowledge tracing. In *Proceedings of the 26th international conference on world wide web*.
- Zhao, W., Xia, J., Jiang, X., & He, T. (2023). A novel framework for deep knowledge tracing via gating-controlled forgetting and learning mechanisms. *Information Processing & Management*, (1).